

Apostila do Curso: Fundamentos de Computação Quântica
Notas de Aula Integradas — Curso de Extensão

Prof. Maxwell Amaral

2026-08-25

Sumário

Capítulo 1

Módulo 1: Panorama e Fundamentos Físicos

Capítulo 2

Aula 01 — Panorama: por que computação quântica?

2.1 Visão geral: da radiação térmica aos qubits

Capítulo 3

Introdução à computação quântica: da radiação térmica aos qubits

3.1 Objetivos da aula

Ao final deste material, o estudante deverá ser capaz de:

- explicar por que alguns experimentos não podiam ser compreendidos pela física clássica;
- distinguir energia contínua de energia quantizada;
- relacionar fótons, ondas de matéria e função de onda;
- interpretar a dupla fenda como uma demonstração de superposição e interferência;
- reconhecer como essas ideias reaparecem no funcionamento de um qubit.

Ideia orientadora: a computação quântica não surgiu diretamente da informática. Ela nasceu de uma longa tentativa de compreender como luz e matéria se comportam em escala microscópica.

3.2 1. O problema da radiação do corpo negro

Um **corpo negro** é um modelo ideal de objeto que absorve toda a radiação que recebe. A melhor realização experimental é uma cavidade aquecida com um pequeno orifício. A radiação que escapa pelo orifício representa muito bem a radiação térmica existente no interior.

Na montagem histórica, a cavidade era aquecida, a radiação saía pelo pequeno orifício e um prisma ou uma rede de difração separava essa radiação por comprimento de onda. Um detector media a intensidade em cada região do espectro.

Todas as faixas de comprimento de onda são emitidas simultaneamente. Quando a temperatura aumenta:

- a intensidade total cresce;
- o pico do espectro se desloca para comprimentos de onda menores;
- o objeto passa de praticamente invisível para vermelho, alaranjado e, em temperaturas maiores, branco.

Isso não significa que o objeto emita uma única cor. Ele emite um espectro contínuo, mas algumas regiões são mais intensas que outras.

3.2.1 Atividade interativa

[Abrir o simulador do corpo negro](#)

Experimente alterar a temperatura e observe:

1. Como muda a intensidade total?

2. Para que lado o pico se desloca?
3. Em temperaturas baixas, em qual região está a maior parte da radiação?

3.2.2 A dificuldade clássica

A física clássica previa que a cavidade deveria emitir energia cada vez maior em frequências muito altas. Essa previsão, chamada posteriormente de **catástrofe do ultravioleta**, não correspondia aos resultados experimentais.

3.3 2. Planck e a quantização da energia

Em 1900, Max Planck propôs que as oscilações nas paredes da cavidade não poderiam trocar qualquer quantidade de energia. A energia seria absorvida ou emitida em pacotes:

$$E = nhf, \quad n = 0, 1, 2, 3, \dots$$

Nessa expressão:

- E é a energia permitida;
- n é um número inteiro;
- h é a constante de Planck;
- f é a frequência da radiação.

A analogia mais simples é a diferença entre uma rampa e uma escada. Na descrição clássica, qualquer valor intermediário seria possível. Na quantização, somente determinados degraus são permitidos.

Em deep learning, “quantização” é uma aproximação de engenharia que representa números usando menos níveis. Na física quântica, os níveis discretos são uma característica do próprio sistema físico.

3.4 3. Como o espectro revela a temperatura

O prisma não mede diretamente a temperatura da luz. Ele separa a radiação em diferentes comprimentos de onda. Medindo a intensidade de cada faixa, construímos a curva espectral.

Para um corpo negro, a lei de Wien relaciona o comprimento de onda do pico à temperatura:

$$\lambda_{\max} T = b,$$

em que $b \approx 2,898 \times 10^{-3} \text{ m} \cdot \text{K}$.

O espectro solar se aproxima do espectro de um corpo negro com temperatura efetiva de aproximadamente 5.800 K, ou cerca de 5.500 °C na fotosfera.

As linhas escuras presentes no espectro solar são linhas de absorção. Elas ajudam a identificar os elementos químicos presentes; a forma geral da curva é que fornece a estimativa de temperatura.

3.5 4. Radiações invisíveis: infravermelho e ultravioleta

3.5.1 Infravermelho

Em 1800, William Herschel separou a luz solar com um prisma e colocou termômetros em diferentes regiões do espectro. Um termômetro posicionado além do vermelho, onde o olho não via luz, também aqueceu. Herschel detectou energia invisível: o **infravermelho**.

O termômetro não media a “temperatura da cor”. Ele funcionava como detector da energia absorvida.

3.5.2 Ultravioleta

Em 1801, Johann Wilhelm Ritter usou cloreto de prata, uma substância que escurece sob a ação da luz. A reação foi ainda mais intensa além do violeta, revelando a radiação que hoje chamamos de **ultravioleta**.

3.6 5. O efeito fotoelétrico e os fótons

No efeito fotoelétrico, luz incide sobre um metal e pode ejetar elétrons. Os resultados mostraram que:

- abaixo de uma frequência mínima, nenhum elétron é ejetado, mesmo aumentando muito a intensidade;
- acima dessa frequência, a emissão ocorre sem atraso perceptível;
- aumentar a frequência aumenta a energia cinética máxima dos elétrons;
- aumentar a intensidade, quando a frequência já é suficiente, aumenta principalmente a quantidade de elétrons emitidos.

Einstein explicou o fenômeno em 1905 tratando a luz como pacotes de energia, posteriormente chamados **fótons**:

$$E_{\text{fóton}} = hf.$$

Para retirar um elétron do metal, o fóton precisa fornecer pelo menos a função trabalho Φ :

$$K_{\text{max}} = hf - \Phi.$$

Uma luz mais intensa possui mais fótons. Porém, se cada fóton tiver energia insuficiente, aumentar sua quantidade não resolve o problema.

3.7 6. Bohr e os níveis de energia do átomo

Em 1913, Niels Bohr propôs que o elétron no átomo só poderia ocupar certos níveis de energia. Ao passar de um nível mais alto para outro mais baixo, o átomo emitiria um fóton:

$$hf = E_i - E_f.$$

O modelo de Bohr foi uma etapa histórica decisiva, embora as “órbitas” não sejam a descrição aceita pela mecânica quântica moderna. Hoje descrevemos o elétron por estados e orbitais, e não como um pequeno planeta em trajetória definida.

3.8 7. De Broglie: matéria também se comporta como onda

Em 1924, Louis de Broglie propôs que uma partícula com momento p possui um comprimento de onda associado:

$$\lambda = \frac{h}{p}.$$

Essa ideia ajuda a compreender por que apenas certos estados são permitidos: os padrões ondulatórios precisam satisfazer condições específicas. Pouco depois, experimentos de difração de elétrons confirmaram o comportamento ondulatório da matéria.

3.9 8. Função de onda e probabilidade

A função de onda, representada por ψ , descreve o estado quântico. Ela não é uma trajetória nem uma onda material comum. Em uma descrição simplificada, o quadrado de seu módulo fornece a densidade de probabilidade:

$$P(x) = |\psi(x)|^2.$$

Antes da medição, a função de onda pode estar distribuída por várias possibilidades. Na detecção, porém, encontramos o elétron inteiro em um ponto. Ao repetir o experimento muitas vezes, os impactos formam a distribuição prevista por $|\psi|^2$.

3.10 9. A dupla fenda: superposição e interferência

Na dupla fenda, podemos representar a amplitude total como a soma das amplitudes associadas às duas alternativas:

$$\psi = \psi_1 + \psi_2.$$

A probabilidade de detecção é:

$$P = |\psi_1 + \psi_2|^2.$$

Esse resultado contém um termo de interferência. Em algumas posições, as amplitudes se reforçam; em outras, cancelam-se. Mesmo enviando elétrons individualmente, os impactos pontuais acumulam gradualmente um padrão de franjas.

3.10.1 Atividade interativa 3D

[Abrir o simulador da dupla fenda com elétrons](#)

No simulador, o movimento é uma representação didática:

- o canhão emite um elétron por vez;
- antes da detecção, mostramos um pacote de onda;
- após a barreira, duas amplitudes representam as alternativas associadas às fendas;
- o anteparo registra apenas um impacto pontual por elétron;
- muitos impactos revelam o padrão de interferência.

O controle **Velocidade da animação** altera apenas o ritmo visual. Não representa diretamente a velocidade física do elétron e não modifica o padrão calculado.

Cuidado conceitual: não devemos imaginar uma bolinha clássica dividindo-se em duas metades. O que se superpõe são amplitudes quânticas; a detecção registra um elétron inteiro.

3.11 10. Da dupla fenda ao qubit

Um bit clássico assume 0 ou 1. Um qubit pode estar em uma superposição:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

com

$$|\alpha|^2 + |\beta|^2 = 1.$$

Ao medir o qubit:

- obtemos 0 com probabilidade $|\alpha|^2$;
- obtemos 1 com probabilidade $|\beta|^2$.

A relação com a dupla fenda é estrutural:

Dupla fenda	Qubit
Alternativa pela fenda 1	Estado $ 0\rangle$
Alternativa pela fenda 2	Estado $ 1\rangle$
Soma de amplitudes	Superposição
Diferença de fase entre caminhos	Fase relativa do qubit
Franjas de interferência	Interferência produzida por portas quânticas
Impacto pontual no anteparo	Resultado clássico da medição

Dentro de um computador quântico normalmente não existe uma dupla fenda física. As alternativas são estados do qubit. As **portas quânticas** controlam amplitudes e fases para produzir interferência: caminhos associados a respostas desejadas podem ser reforçados e outros podem ser reduzidos.

3.11.1 Exemplo mínimo: porta Hadamard

Se o qubit começa em $|0\rangle$, a porta Hadamard cria a superposição:

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

Aplicando a mesma porta novamente:

$$H\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right) = |0\rangle.$$

As amplitudes não foram apenas “sorteadas”: elas interferiram. Essa possibilidade de manipular interferência é uma das principais fontes do poder dos algoritmos quânticos.

3.12 11. Sequência sugerida para apresentação

1. Apresente a cavidade de corpo negro e deixe a turma explorar a temperatura.
2. Mostre por que a previsão clássica falhava.
3. Introduza os pacotes de Planck e a analogia da escada.
4. Use o efeito fotoelétrico para transformar a quantização em uma explicação física da luz.
5. Passe aos níveis de Bohr e à hipótese de de Broglie.
6. Apresente $|\psi|^2$ como mapa de probabilidades.
7. Explore a dupla fenda no simulador, primeiro lentamente e depois acumulando impactos.
8. Faça a correspondência entre duas fendas, duas amplitudes e os estados $|0\rangle$ e $|1\rangle$.
9. Termine com a porta Hadamard como primeiro exemplo de interferência controlada.

3.13 12. Perguntas para discussão

1. Por que aumentar a intensidade de uma luz vermelha pode não produzir efeito fotoelétrico?
2. Qual é a diferença entre a função de onda e a probabilidade de detecção?
3. Se cada elétron produz apenas um ponto, como surge um padrão de interferência?
4. O controle de velocidade do simulador muda a física ou apenas a visualização?
5. Em que sentido um qubit se parece com a dupla fenda?
6. Por que dizer que o qubit é “zero e um ao mesmo tempo” é útil, mas incompleto?

3.14 13. Exercício de consolidação

Peça aos estudantes que completem a frase:

“Na dupla fenda, as amplitudes associadas aos caminhos interferem antes da medição. Em um computador quântico, as amplitudes associadas aos estados _____ interferem por meio de _____, e a medição produz _____.”

Resposta esperada: $|0\rangle$ e $|1\rangle$; **portas quânticas; um resultado clássico.**

3.15 Resumo

A trajetória conceitual desta aula é:

radiação térmica → quantização → fótons → níveis de energia → ondas de matéria → função de onda → superposição → interferência → qubits.

O elemento unificador é a amplitude quântica. Ela pode ser combinada, adquirir fase e interferir. A computação quântica transforma esse comportamento, observado inicialmente em experimentos fundamentais, em uma ferramenta controlável para processar informação.

3.16 A natureza da computação quântica

Referência: https://brilliant.org/courses/quantum-computing/introduction-108/the-nature-of-computation-2/?from=icp_node&from_llp=science

Bem-vindo à Computação Quântica! Nossa área de atuação promete revolucionar tudo, desde a quebra de códigos até o desenvolvimento de medicamentos e o aprendizado de máquina. Com tanta expectativa, é fácil se perder maravilhado com as possibilidades, sem compreender o que a computação quântica realmente é.

Nosso foco é aprender como explorar as leis da mecânica quântica para realizar cálculos. Na maior parte do tempo, nos concentraremos nas estruturas da mecânica quântica que são úteis para esse fim — e permaneceremos em feliz ignorância sobre a física quântica em detalhes.

Ao final deste curso, você terá uma visão deslumbrada dos algoritmos e da tecnologia que fundamentam o que você lê nas notícias sobre computação quântica. Mas antes disso, precisamos entender o panorama geral, o que podemos fazer respondendo a duas perguntas:

1. Qual é a natureza da computação quântica, e
2. Por que os cientistas acreditam que isso levará a avanços significativos na resolução de diversos problemas importantes?

![[A natureza da computação quântica - image 002.png|500x411]]

Fótons individuais que passam por uma fenda dupla podem ser detectados em qualquer lugar, mas cumulativamente formarão uma distribuição de probabilidade.

No século XXI, as previsões **determinísticas** da mecânica clássica não conseguiam explicar novos experimentos.

Uma nova teoria se fez necessária para descrever os resultados de experimentos realizados com objetos quânticos: fótons, elétrons e outras partículas microscópicas. A mecânica quântica abandona as previsões determinísticas e, em vez disso, adota o conceito de **probabilidade**. Em outras palavras, a mecânica quântica pode prever a distribuição estatística dos resultados de **muitos** experimentos, embora qualquer experimento individual seja um evento aleatório.

Como a computação quântica é uma história sobre a subordinação de objetos quânticos, é apropriado começar nossa exploração com um jogo simples. Vamos jogá-lo com fótons.

Este simples jogo de azar é conhecido no mundo todo. No Japão, é chamado de *pachinko*, e em programas de jogos americanos é conhecido como *Plinko*. Para matemáticos e físicos, às vezes é chamado de tabuleiro de Galton.

Independentemente do nome que lhe dermos, este jogo é uma demonstração simples de como as probabilidades podem surgir de um objeto mecânico. Cada compartimento na parte inferior está associado a um prêmio diferente, e a emoção está em observar como o seu destino é determinado pelos quiques aleatórios da bola nos pinos.

Nesta lição, vamos recriar o jogo com **objetos quânticos**: um fóton no lugar da bola e divisores de feixe em cada canto.

![[plinko-vs-lasers.mp4]]

A cada quique, a bola e o fóton se comportam de maneira semelhante: ambos têm a opção de seguir dois caminhos: para a direita ou para a esquerda, cada um com **igual probabilidade**.

Qual é o resultado mais provável desses jogos?

Com um único fóton, o jogo quântico se comporta de maneira muito semelhante à versão mecânica com a bola quicando; quanto mais caminhos possíveis levarem a um determinado resultado, maior a probabilidade de ele ocorrer.

Mas se dispararmos dois fótons no topo da grade, as coisas ficam mais interessantes. Quando dois ou mais fótons se encontram em algum ponto da grade, eles podem **interferir** uns com os outros.

A interferência pode ter aparências muito diferentes dependendo dos detalhes: os dois fótons podem se somar construtivamente e aparecer como um sinal mais forte, ou podem se cancelar mutuamente como se não houvesse fótons.

![[quantum-interference.gif]]

Graças a essa interferência, não temos maneira melhor de descobrir onde os fótons provavelmente irão parar do que traçar **todos os caminhos** que cada fóton pode percorrer e calcular os efeitos de interferência nos pontos de encontro.

Por mais complexo que isso seja (muito complexo), não estamos aqui preocupados com o mistério da física, mas sim com o último ponto:

Para prever o estado final de um objeto quântico, precisamos levar em conta todos os caminhos que ele poderia ter percorrido para chegar lá.

Quão difícil é prever **computacionalmente** os resultados deste jogo quântico?

Se dispararmos dois fótons contra o topo, cada um seguirá um caminho probabilístico através da grade. Dependendo da fonte de fótons e do caminho que percorrem, dois fótons que chegam ao mesmo ponto podem se combinar e parecer dois fótons, ou se cancelar completamente.

Simular este jogo em um computador é uma tarefa gigantesca: a única maneira que conhecemos para calcular a probabilidade de encontrar um fóton em cada marcador é traçar todas as **combinações** possíveis de trajetórias para cada fóton e calcular os efeitos de interferência nos pontos de encontro dos fótons (indicados aqui em vermelho).

![[photon-paths-2.gif]]

Neste caso simples com duas fileiras de divisores de feixe e dois fótons, cada fóton pode seguir um de quatro caminhos, resultando em **dezesesseis** combinações diferentes de trajetórias para os cálculos de interferência. À medida que o jogo se torna mais complexo, mais e mais trajetórias acabarão se encontrando e interferindo.

![[lasers.mp4]]

![[probabilidade-relativa.svg]]

Para cada uma das combinações de trajetórias, calcular a interferência entre elas é rápido e fácil, e leva t_c segundos.

Quanto tempo leva para calcular a distribuição de fótons para um jogo quântico de 3 camadas - com dois fótons?

Simular esse jogo simples parece exigir muitos cálculos. **E se simplesmente o construíssemos?**

Com uma grade divisora de feixe em mãos, podemos disparar dois fótons na parte superior e usar contadores de fótons para determinar experimentalmente a distribuição na parte inferior.

![[divisora de feixes.png]]

Com apenas um par de fótons por experimento, obter uma estimativa confiável da distribuição no detector envolve amostrar e calcular a média de milhões de fótons. Para cada medição, o tempo de amostragem seria igual ao tempo que os fótons levam para atravessar a lacuna l do primeiro divisor de feixe até a fileira de contadores de fótons à velocidade da luz. Para um único divisor de feixe com dois detectores, essa amostragem de fótons leva tempo t_q .

![[lasers2.mp4]]

![[relative-count.svg]]

Quanto tempo leva para uma solução experimental em 3 camadas para este jogo quântico?

A cada camada de divisores de feixe que adicionamos ao jogo, o número de caminhos a serem considerados em nossa simulação computacional aumenta por um fator de 2. Em geral, um jogo de n camadas tem 2^n diferentes caminhos que cada fóton pode seguir.

Mas para calcular a interferência, consideramos **todas as combinações possíveis** destes 2^n trajetórias para dois fótons. Essa abordagem requer o cálculo da interferência para $(2^n)^2 = 2^{2n}$ caminhos, que levariam pelo menos $2^{2n} \cdot t_c$ tempo.

![[n-layers.png]]

Quanto tempo durará uma medição de contagem de fótons em um jogo de n camadas **experimentalmente** ?

O tempo necessário para cada um dos nossos métodos calcular o padrão de interferência aumenta a taxas diferentes conforme o número de n camadas aumenta. Para uma única camada, o tempo que um computador leva para calcular a probabilidade de dois caminhos é provavelmente ínfimo — muito menos de um segundo, enquanto a média de milhões de medições de fótons individuais pode levar vários segundos.

Mas como n aumenta o tempo necessário para o cálculo clássico, $T_{classic} = 2^{2n}t_c$ aumenta exponencialmente com n à medida que o número de caminhos possíveis explode. No entanto, o tempo necessário para o experimento quântico $T_q = nt_q$ aumenta apenas linearmente em n . Isso ocorre porque os objetos quânticos no experimento não têm a obrigação de calcular trajetórias de acordo com as regras da mecânica quântica; eles obedecem à mecânica quântica naturalmente.

Para avaliar a importância desse aumento de velocidade, podemos assumir, generosamente t_q para ser aproximadamente 10s (com todas as medições repetidas) e t_c ser 100 s.

![[quantum-layers.mp4]]

Quantas camadas de divisores de feixe serão necessárias para que nosso cálculo clássico seja superado pelo experimento?

Deixamos claro que nosso cálculo de busca de caminho é o tipo de coisa que podemos computar em um computador clássico. Por outro lado, temos tratado a abordagem do divisor de feixe mais como um experimento de laboratório, não como um cálculo. Mas ambos os métodos produzem a mesma informação, então, se um é um cálculo, o outro também é.

![[classical-quantum-comparation.svg]]

A diferença é que o experimento obtém o resultado muito mais rapidamente. Dessa forma, podemos ver que encontrar o padrão de interferência para o problema das camadas é **difícil** de computar classicamente, porque os recursos necessários crescem exponencialmente à medida que... aumenta, mas é bastante fácil de calcular “quanticamente” com um experimento.

Usamos aspas, mas essa é a verdade fundamental do que acontece na computação quântica: empregar diretamente objetos quânticos para realizar um cálculo específico. Na medida em que pudermos mapear e moldar nossos problemas computacionais em objetos quânticos, poderemos usar isso a nosso favor. Se um problema tiver um aspecto quântico inerente, como encontrar o padrão de interferência, então esperamos um aumento de velocidade.

E essa é a questão: “Quais dos nossos problemas computacionais clássicos têm um aspecto quântico e como o percebemos quando ele está presente?”

Para termos uma ideia do que significa mapear problemas em sistemas físicos, podemos considerar a importância computacional de conceitos mais familiares da física.

Considere esta escultura, na qual o tecido verde entra em contato com todos os “limites” formados pelos círculos de metal.

![[green-void-13.jpg]]

Vazio Verde, LAVA (LABORATÓRIO PARA ARQUITETURA VISIONÁRIA) www.lava.net

Na arquitetura, assim como no estudo dos buracos negros, um problema fundamental é encontrar formas que utilizem a menor quantidade de material possível para se ajustarem às restrições. De fato, a forma que vemos acima é ideal nesse sentido: é impossível encontrar qualquer outra forma tão suave que entre em contato com os anéis de metal utilizando menos material.

Encontrar essa superfície por meio de computação não é simples e requer algoritmos baseados em matemática sofisticada. Existe uma maneira mais fácil?

Que fenômeno seria capaz de “calcular” essa superfície em tempo real, usando as leis da física?

Sistemas naturais, como bolhas de sabão, tendem a se movimentar até que sua energia se estabilize. Prepare-as, dê uma sacudida e espere até que se acalmem.

Para uma película de sabão, a energia é igual à sua área superficial total S vezes a sua tensão superficial γ . Essa é basicamente a energia necessária para manter a camada de sabão esticada:

$$E = \gamma \times S$$

Imagine que criamos uma forma com arame, mergulhamos nela em um banho de água com sabão, a retiramos e deixamos a mistura assentar. Esse cálculo complexo é realizado em tempo real pelas leis da física, enquanto que, mesmo em um computador moderno e potente usando um solucionador não linear, levaria vários segundos para ser executado.

![[quantum-image-5.png]]

Para a maioria das pessoas, o poder computacional da água do banho é essencialmente zero; mas se você tiver um problema muito específico para resolver, ela pode ser superior a um computador moderno.

![[animatedbubblehelix8.gif]]

Em última análise, as aplicações mais óbvias dos computadores quânticos parecem triviais — naturalmente, um computador construído com fótons será eficaz na simulação de fótons. Os computadores quânticos não são uma panaceia para a computação em geral, assim como água com sabão não é para a física dos buracos negros, mas essa aparente trivialidade esclarece o nosso desafio.

![[bubble-image-2.svg]]

O problema da superfície mínima pode ser facilmente mapeado para a energia superficial de bolhas de sabão, o que está de acordo com a física clássica. Podemos **aproveitar a natureza** para realizar o cálculo para nós, expressando-o na linguagem da mecânica clássica.

Em um sentido real, os computadores quânticos não realizam cálculos, mas simplesmente operam de acordo com as leis da física. A extensão em que a computação quântica pode revolucionar a computação depende de encontrarmos maneiras de reformular problemas computacionais em sistemas quânticos equivalentes.

Para alguns problemas, como simular as ligações químicas em uma molécula, isso é relativamente simples. Para outros problemas, como pesquisar em um banco de dados ou fatorar um número inteiro, encontrar a ponte é muito mais complexo. É por isso que dedicaremos grande parte do próximo capítulo ao aprendizado das leis relevantes da mecânica quântica, mantendo-nos próximos à estrutura definida pela computação clássica.

Entretanto, existe uma objeção quando a aleatoriedade (não determinismo) e o superposicionamento relatado na mecânica quântica padrão, conforme pode ser visto em [[Contradições e realidade na Mecânica Quântica]]

Capítulo 4

Módulo 2: Da Física Clássica à Quântica

Capítulo 5

Aula 02 — Da Física Clássica à Quântica: Corpo Negro, Efeito Fotoelétrico e Bohr

Capítulo 6

Módulo 2 — Radiação do corpo negro e quantização

6.1 A cavidade negra

Um corpo negro ideal absorve toda a radiação incidente. Na prática, usa-se uma cavidade aquecida com um pequeno orifício. A radiação interna sofre muitas absorções e emissões, e uma pequena parte escapa pelo furo.

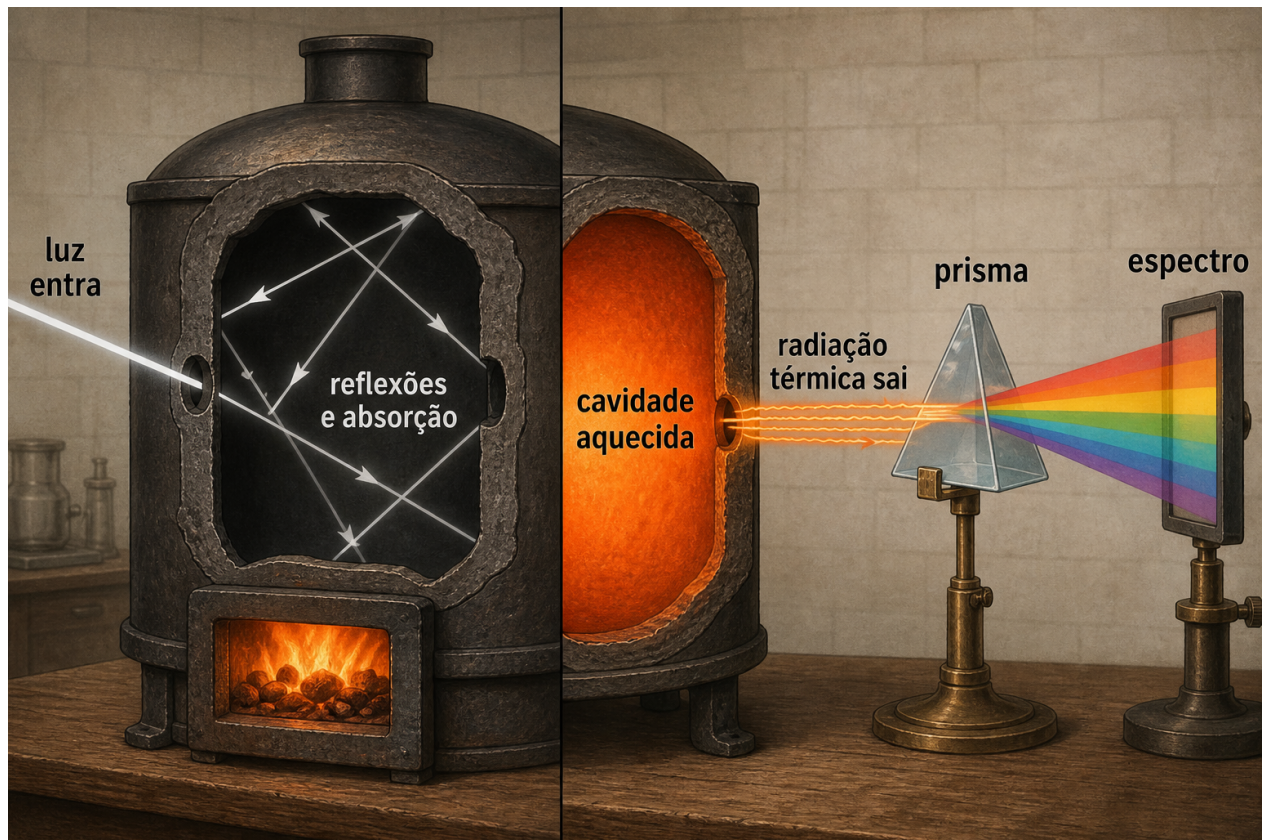


Figura 6.1: Experimento da cavidade

Passando a radiação por um prisma ou uma rede de difração e medindo sua intensidade, obtemos um espectro contínuo. Todas as faixas são emitidas simultaneamente; a temperatura muda a intensidade relativa e a posição do pico.

[Explorar o simulador do corpo negro](#)

6.2 Temperatura pelo espectro

A lei de Wien relaciona o pico do espectro à temperatura:

$$\lambda_{\max} T = b.$$

O espectro do Sol se aproxima do de um corpo negro a aproximadamente 5.800 K.

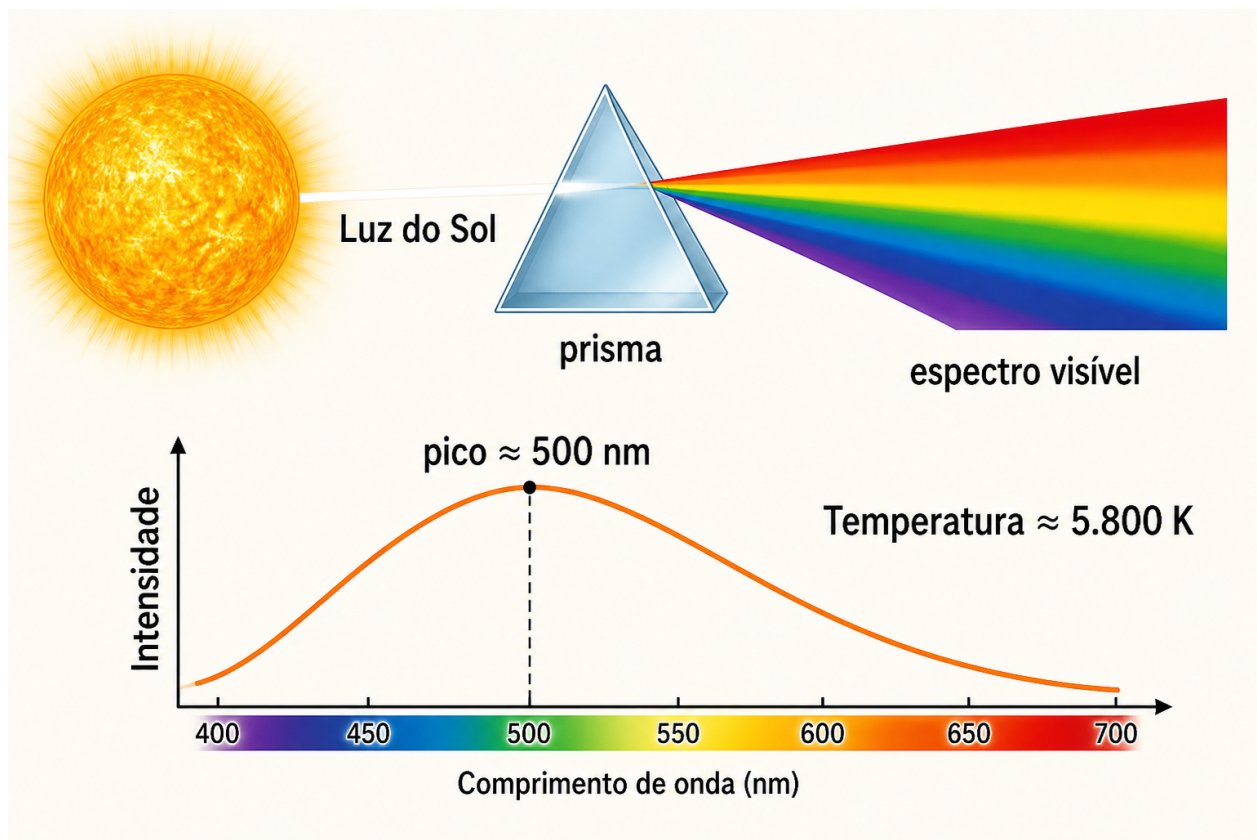


Figura 6.2: Espectro solar e temperatura

6.3 A crise clássica

A teoria clássica previa energia ilimitada em frequências altas, a chamada catástrofe do ultravioleta. Planck resolveu o problema propondo que os osciladores das paredes trocavam energia em quantidades discretas:

$$E_n = nhf, \quad n = 0, 1, 2, \dots$$



Figura 6.3: Quantização na cavidade

6.4 Relação com a computação quântica

Um qubit físico também possui estados de energia controláveis e transições quantizadas. Em qubits supercondutores, por exemplo, pulsos de micro-ondas manipulam transições entre dois níveis escolhidos para representar $|0\rangle$ e $|1\rangle$. A constante de Planck e a relação entre energia e frequência estão no centro desse controle.

6.5 Atividade

No simulador, aumente a temperatura e registre como variam intensidade, cor aparente e posição do pico. Explique por que não sai apenas uma cor pelo furo.

Capítulo 7

Efeito

Capítulo 8

Módulo 3 — Efeito fotoelétrico e fótons

8.1 Experimento

Luz incide sobre uma superfície metálica. Quando a frequência é suficiente, elétrons são ejetados e podem ser coletados por um circuito. Hallwachs usou uma placa de zinco e luz ultravioleta; outros metais, como sódio e cézio, respondem a frequências menores.

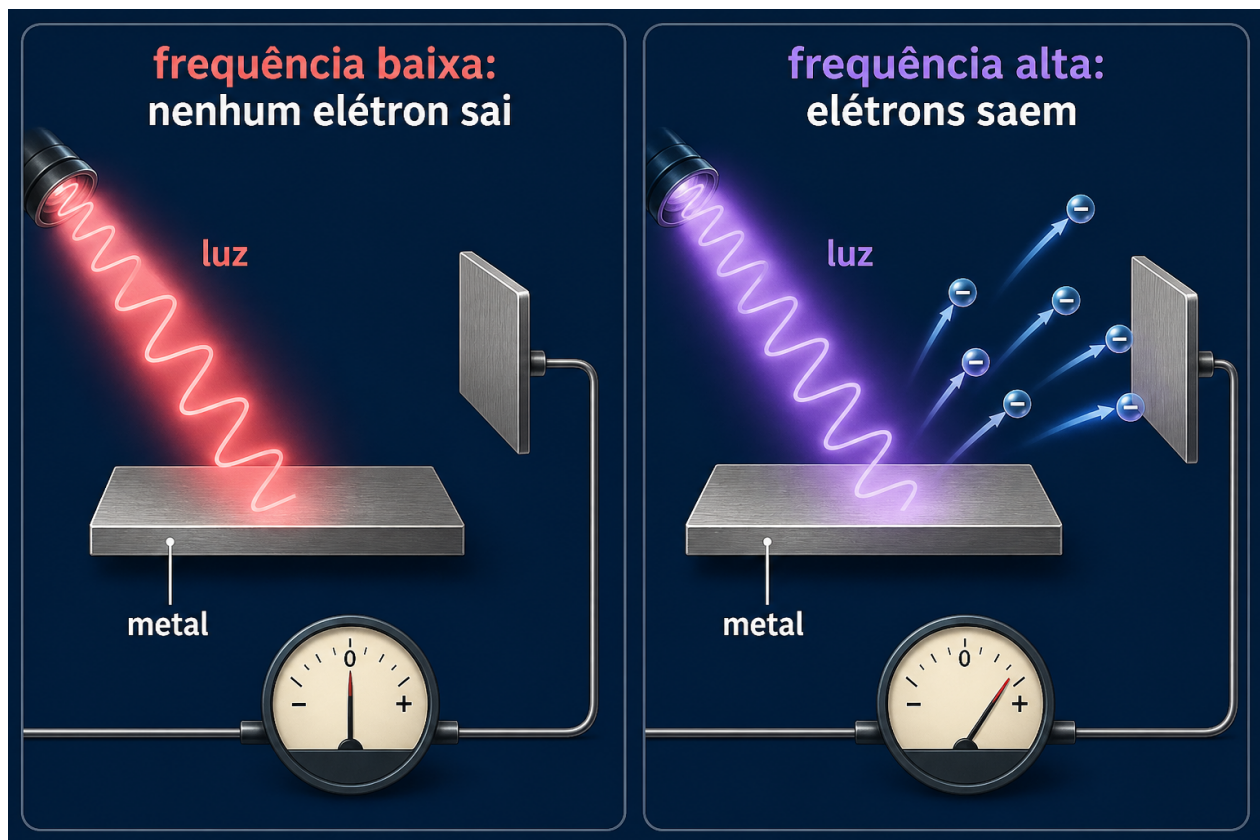


Figura 8.1: Efeito fotoelétrico

Os resultados fundamentais são:

- existe uma frequência mínima para cada material;
- abaixo dela, aumentar a intensidade não ejeta elétrons;
- acima dela, maior frequência produz elétrons com maior energia cinética;
- maior intensidade significa principalmente mais elétrons emitidos.

Einstein explicou o fenômeno atribuindo à luz pacotes de energia:

$$E_{\text{fóton}} = hf,$$

$$K_{\text{max}} = hf - \Phi,$$

onde Φ é a energia mínima necessária para liberar o elétron.

8.2 Relação com a computação quântica

O experimento mostra que interações quânticas individuais produzem eventos discretos. Computadores quânticos também terminam em medições discretas: cada execução devolve bits clássicos, embora o estado anterior seja descrito por amplitudes. Detectores de fótons e interfaces luz-matéria também são componentes importantes da computação e comunicação quânticas fotônicas.

8.3 Cuidado conceitual

Um elétron não sai necessariamente numa única direção fixa. A direção depende do material, da luz e das colisões internas; o aparelho coleta uma distribuição de elétrons.

Capítulo 9

Bohr

Capítulo 10

Módulo 4 — Bohr e níveis de energia

Bohr propôs, em 1913, que o elétron só poderia ocupar determinados níveis de energia. Uma transição entre níveis envolve emissão ou absorção de um fóton:

$$hf = |E_f - E_i|.$$

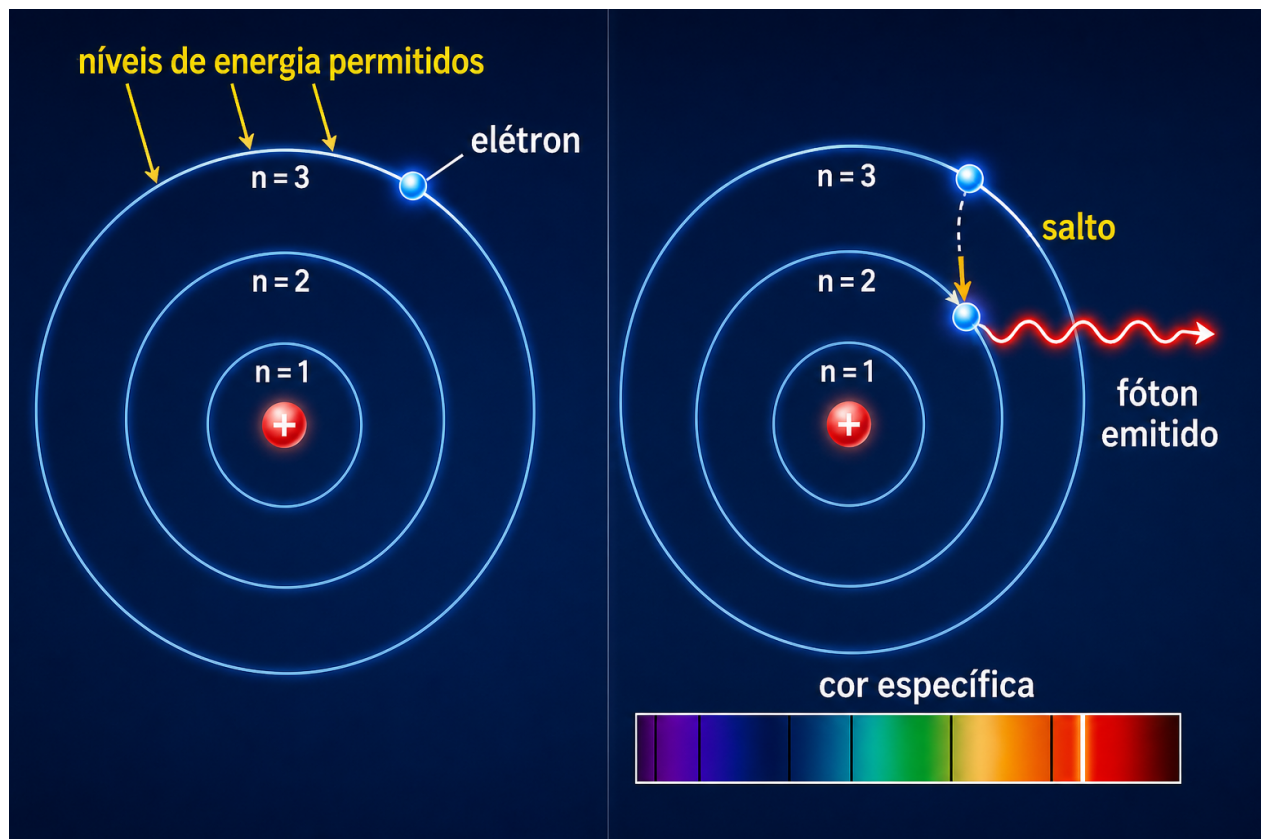


Figura 10.1: Modelo de Bohr

Isso explicou por que átomos emitem linhas espectrais específicas, em vez de todas as cores possíveis.

10.1 Limite do modelo

A imagem de órbitas circulares é uma etapa histórica, não a descrição moderna. Na mecânica quântica, elétrons são descritos por estados e orbitais, não por trajetórias planetárias definidas.

10.2 Relação com a computação quântica

Um qubit escolhe dois estados distinguíveis de um sistema quântico e os chama de $|0\rangle$ e $|1\rangle$. Pulsos com frequência ressonante à diferença de energia permitem controlar transições. O desafio prático é preservar esses estados contra ruído e evitar transições indesejadas para outros níveis.

10.3 Pergunta de verificação

Por que um átomo produz linhas de cores específicas em vez de um arco-íris contínuo?

Capítulo 11

Módulo 3: Ondas de Matéria e Dupla Fenda

Capítulo 12

Aula 03 — Ondas de Matéria, Função de Onda e Dupla Fenda

Capítulo 13

Módulo 5 — De Broglie e ondas de matéria

De Broglie propôs que uma partícula com momento p possui comprimento de onda:

$$\lambda = \frac{h}{p}.$$

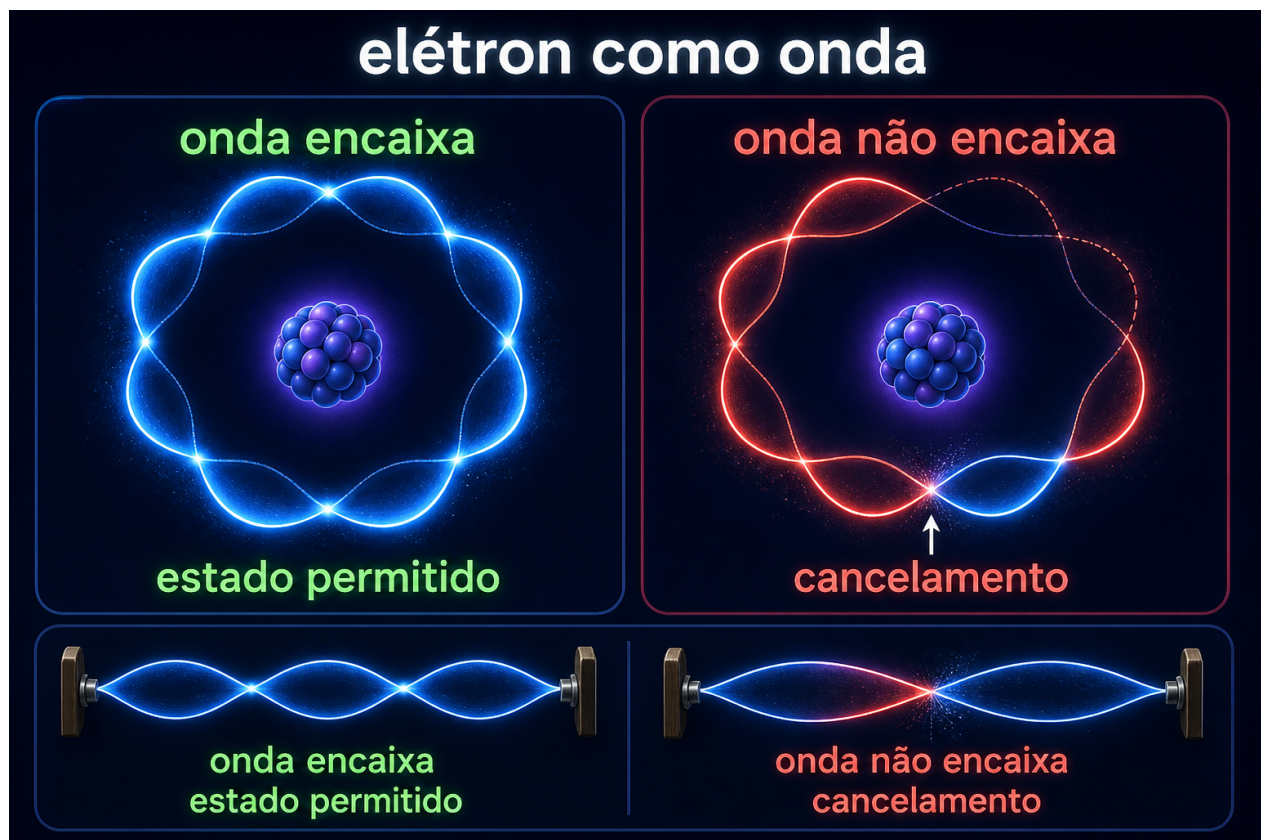


Figura 13.1: Ondas de matéria

Padrões ondulatórios permitidos precisam satisfazer condições de contorno, de modo semelhante a uma onda estacionária numa corda. Experimentos posteriores de difração de elétrons confirmaram que matéria pode produzir interferência.

13.1 O que é difração?

É o espalhamento característico de uma onda ao atravessar aberturas ou contornar estruturas. Regiões de reforço e cancelamento formam um padrão que partículas clássicas, seguindo apenas trajetórias definidas, não produziram.

13.2 Relação com a computação quântica

Qubits não são bolinhas clássicas. Seus estados têm amplitudes e fases, propriedades matemáticas ondulatórias. A computação quântica usa precisamente a possibilidade de controlar essas fases e fazer amplitudes interferirem.

13.3 Atividade

Compare uma órbita clássica, uma onda que fecha sobre si mesma e outra que não fecha. Discuta por que a quantização pode surgir das condições impostas à onda.

Capítulo 14

Módulo 6 — Função de onda e dupla fenda

14.1 Função de onda

A função de onda ψ descreve o estado quântico. Ela não fornece uma trajetória. O módulo ao quadrado fornece a densidade de probabilidade:

$$P(x) = |\psi(x)|^2.$$

Cada detecção registra uma partícula inteira em um ponto. Muitas repetições revelam a distribuição prevista.

14.2 Experimento da dupla fenda

Com duas fendas abertas, a amplitude total é:

$$\psi = \psi_1 + \psi_2,$$

e a probabilidade é:

$$P = |\psi_1 + \psi_2|^2.$$

O termo de interferência cria regiões de reforço e cancelamento. Mesmo emitindo elétrons um por vez, os pontos acumulados formam franjas.

[Explorar o simulador 3D com elétrons](#)

No simulador:

- o canhão emite elétrons;
- o pacote representa a evolução ondulatória;
- duas amplitudes aparecem depois das fendas;
- cada elétron deixa somente um ponto no anteparo;
- o controle de velocidade muda o ritmo da animação, não a física.

A animação é um modelo didático. Não representa uma pequena esfera dividindo-se fisicamente. O que passa a ser combinado são amplitudes quânticas.

14.3 Relação com a computação quântica

A dupla fenda contém três ingredientes essenciais: superposição de alternativas, fase relativa e interferência. Um algoritmo quântico organiza esses mesmos ingredientes no espaço de estados dos qubits.

14.4 Experiência complementar: informação de caminho

Se o aparelho obtém informação capaz de distinguir por qual fenda o elétron passou, a interferência diminui ou desaparece. Isso introduz medição, correlação com o ambiente e decoerência — temas decisivos para computadores quânticos reais.

Capítulo 15

Módulo 4: O Qubit

Capítulo 16

Aula 04 — O Qubit: Fundamentos Matemáticos da Computação Quântica

Capítulo 17

Fundamentos Matemáticos da Computação Quântica

Bem-vindo! Este notebook foi criado para ajudá-lo a entender os **conceitos matemáticos básicos** necessários para trabalhar com computação quântica.

17.1 O que você vai aprender:

1. **Estados quânticos** - Como representar qubits matematicamente
2. **Portas quânticas** - Operações que transformam estados quânticos
3. **Superposição** - O “poder” dos computadores quânticos
4. **Visualizações** - Esfera de Bloch e outras representações visuais

17.2 Para quem é este notebook?

Se você conhece: - Álgebra linear básica (vetores e matrizes) - Números complexos (pelo menos saber que existem) - Python básico

Então você está pronto para começar! Não se preocupe se alguns conceitos parecerem estranhos no início - isso é normal em computação quântica.

Dica: Execute cada célula na ordem para acompanhar os exemplos interativos!

Observação Para instalar as bibliotecas abaixo, você deve abrir um terminal e rodar `uv sync` na raiz deste repositório.

```
# Importando as bibliotecas necessárias
# Não se preocupe se não entender todas agora - vamos usá-las ao longo do notebook

import numpy as np                # Para operações com matrizes e vetores
import sympy as sp                # Para matemática simbólica (fórmulas bonitas)
from sympy import Matrix, symbols # Para criar matrizes e símbolos matemáticos
from IPython.display import Markdown # Para exibir fórmulas formatadas
from qiskit.quantum_info import Statevector # Para trabalhar com estados quânticos
from matplotlib import pyplot as plt # Para criar gráficos
from qiskit.visualization import plot_bloch_multivector, plot_state_qsphere # Visualizações quânticas
```

17.3 Estados Quânticos Básicos

Na computação clássica, trabalhamos com **bits** que podem ser 0 ou 1.

Na computação quântica, trabalhamos com **qubits** que podem estar em: - **Estado $|0\rangle$** (lê-se “ket zero”) - equivalente ao bit clássico 0 - **Estado $|1\rangle$** (lê-se “ket um”) - equivalente ao bit clássico 1 - **Superposição de $|0\rangle$ e $|1\rangle$** - aqui está a mágica! O qubit pode estar em “ambos” estados simultaneamente

17.3.1 Notação matemática:

Representamos esses estados como **vetores coluna** em “ket psi”. Em geral:

$$|\psi\rangle = \begin{bmatrix} \text{amplitude para estado } \alpha \\ \text{amplitude para estado } \beta \end{bmatrix}$$

Essa amplitude pode variar de -1 a 1 em números reais ou é um número complexo cujo módulo (valor absoluto) é menor ou igual a 1 .

Assim:

- Estado $|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$
 - **Primeira posição (1)**: Amplitude para o estado $|0\rangle$
 - **Segunda posição (0)**: Amplitude para o estado $|1\rangle$
 - Interpretação: 100% de probabilidade de medir 0, 0% de probabilidade de medir 1
- Estado $|1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$
 - **Primeira posição (0)**: Amplitude para o estado $|0\rangle$
 - **Segunda posição (1)**: Amplitude para o estado $|1\rangle$
 - Interpretação: 0% de probabilidade de medir 0, 100% de probabilidade de medir 1

Regra geral: Para qualquer qubit $|\psi\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$: - O valor α (primeira linha) representa a amplitude do estado $|0\rangle$ - O valor β (segunda linha) representa a amplitude do estado $|1\rangle$ - As probabilidades são: $P(0) = |\alpha|^2$ e $P(1) = |\beta|^2$ - Sempre vale: $|\alpha|^2 + |\beta|^2 = 1$ (normalização)

Por que vetores? Porque podemos aplicar transformações matemáticas (matrizes) neles, e isso é exatamente o que as portas quânticas naturalmente fazem!

```
# Vamos criar esses estados básicos usando SymPy (biblioteca de matemática simbólica)
# Isso nos permite trabalhar com fórmulas matemáticas de forma elegante

# Estado |0 - primeiro número é 1, segundo é 0
ket_0 = Matrix([[1],
                [0]])

# Estado |1 - primeiro número é 0, segundo é 1
ket_1 = Matrix([[0],
                [1]])

# Exibir de forma bonita com LaTeX (notação matemática profissional)
display(Markdown(f"$$$\\ket{{0}} = \\{sp.latex(ket_0)}$$$"))
display(Markdown(f"$$$\\ket{{1}} = \\{sp.latex(ket_1)}$$$"))

print("\n Estados básicos criados com sucesso!")
```

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$|1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Estados básicos criados com sucesso!

Estados básicos criados com sucesso!

```
# Agora vamos criar os MESMOS estados usando NumPy (só para fins de comparação de bibliotecas)
# NumPy é mais eficiente para cálculos numéricos (com números reais)

# Estado |0 usando NumPy
q_zero = np.array([
    [1], # Probabilidade 100% de medir 0
    [0]  # Probabilidade 0% de medir 1
])

# Estado |1 usando NumPy
q_one = np.array([
    [0], # Probabilidade 0% de medir 0
    [1]  # Probabilidade 100% de medir 1
])

print("=== Estado |0 ===")
print(q_zero)
print("\n=== Estado |1 ===")
print(q_one)

print("\n Dica: Esses vetores representam a 'probabilidade' de medir cada resultado!")
```

```
=== Estado |0 ===
[[1]
 [0]]
```

```
=== Estado |1 ===
[[0]
 [1]]
```

Dica: Esses vetores representam a 'probabilidade' de medir cada resultado!

```
=== Estado |0 ===
[[1]
 [0]]
```

```
=== Estado |1 ===
[[0]
 [1]]
```

Dica: Esses vetores representam a 'probabilidade' de medir cada resultado!

17.4 Superposição: o coração da Computação Quântica

Aqui está onde a mágica acontece! Um qubit pode estar em uma **combinação** dos estados $|0\rangle$ e $|1\rangle$:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Onde: - $|\psi\rangle$ (lê-se “ket psi”) é o estado do qubit - α e β são **números complexos** chamados de **amplitudes** - **Regra de ouro**: $|\alpha|^2 + |\beta|^2 = 1$ (aqui também essas probabilidades precisam somar 100%!).

17.4.1 Como interpretar as amplitudes:

- $|\alpha|^2$ = probabilidade de medir **0**
- $|\beta|^2$ = probabilidade de medir **1**

17.4.2 Exemplo visual:

Se $\alpha = \frac{1}{\sqrt{2}}$ e $\beta = \frac{1}{\sqrt{2}}$?

“Montando” a combinação dos estados $|0\rangle$ e $|1\rangle$, ficará assim:

$$|\psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

Se quiser, pra ficar mais claro, podemos substituir os kets pelos seus vetores, até para ficar mais intuitivo e você sempre imaginar que por trás desses kets, existem, na verdade, vetores:

$$|\psi\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \frac{1}{\sqrt{2}} \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Assim, não precisamos sempre usar vetores, mas é bom saber que eles estão lá por trás.

Agora, calculando as probabilidades:

- Probabilidade de medir 0: $\left(\frac{1}{\sqrt{2}}\right)^2 = \frac{1}{2} = 0.50 = 50\%$
- Probabilidade de medir 1: $\left(\frac{1}{\sqrt{2}}\right)^2 = \frac{1}{2} = 0.50 = 50\%$

Em vetores:

$$|\psi\rangle = \alpha \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \beta \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Simples assim! O primeiro número é a amplitude para $|0\rangle$, o segundo para $|1\rangle$.

```
# Vamos criar um estado genérico em superposição!
# Usamos símbolos (alpha) e (beta) para representar valores desconhecidos

alpha, beta = symbols('alpha beta', complex=True) # Definindo e como números complexos
psi = alpha * ket_0 + beta * ket_1                 # Estado genérico em superposição

print("Estado quântico genérico criado:")

display(Markdown(f"$$$\\ket{{{\\psi}}} = {sp.latex(psi)}$$$"))

print("\n Isso significa:")
print("    - Com probabilidade  $|\alpha|^2$ , medimos 0")
```

```
print("    - Com probabilidade  $| \cdot |^2$ , medimos 1")
print("    - Antes da medição, o qubit está em AMBOS os estados!")
```

Estado quântico genérico criado:

$$|\psi\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

Isso significa:

- Com probabilidade $| \cdot |^2$, medimos 0
- Com probabilidade $| \cdot |^2$, medimos 1
- Antes da medição, o qubit está em AMBOS os estados!

Estado quântico genérico criado:

Isso significa:

- Com probabilidade $| \cdot |^2$, medimos 0
- Com probabilidade $| \cdot |^2$, medimos 1
- Antes da medição, o qubit está em AMBOS os estados!

17.5 E o que é esse termo “superposição”?

Superposição é o fenômeno quântico onde um qubit pode existir em múltiplos estados ao mesmo tempo. Diferente dos bits clássicos que são ou 0 ou 1, um qubit em superposição pode ser uma combinação de ambos os estados.

Como assim? Eu estou acostumado a pensar que algo é 0 ou 1, não ambos ao mesmo tempo!

Imagine uma moeda girando no ar (esse é um comparativo clássico da literatura!). Enquanto está girando, ela não é nem cara nem coroa - está em um estado de superposição de ambos os lados. Só quando a moeda cai e você olha para ela é que ela “decide” ser cara ou coroa. Então, olhar para a moeda é como medir um qubit: você força ele a escolher um estado específico (0 ou 1), fazendo-o parar de girar.

Obviamente, se você for teimoso, vai dizer que os dois não são cara e coroa ao mesmo tempo, pois se você usar uma supercâmera de altíssima velocidade, verá que a moeda está em um estado definido a cada instante. Em câmera super lenta, você verá que a moeda ou está em cara, ou em coroa. OK, teimoso, você está certo! Porém, entretanto, todavia...

No mundo quântico, se essa moeda existisse - vamos chamar de moeda quântica - ele seria cara e coroa ao mesmo tempo. Somente quando você a observasse é que poderia saber se é cara ou coroa. Na visualização, o movimento de giro é destruído ao ser observado. A moeda cai e para. Você perdeu a informação do movimento (superposição) e ficou apenas com o resultado estático (0 ou 1), e só a partir daí você sabe o resultado final. Estranho não é? Bem-vindo ao mundo quântico!

O que acontece: O qubit (o objeto físico, como um elétron ou átomo) geralmente não é destruído (a menos que seja um fóton sendo absorvido por um detector). O que é “destruído” irremediavelmente é a informação da superposição (a parte quântica, o giro). O qubit continua lá, mas agora ele é apenas um bit clássico chato (está parado em cara ou coroa)

Entender isso (e aceitar) é crucial para compreender como os computadores quânticos funcionam e por que eles são tão poderosos para certas tarefas. Daqui para frente, sempre que falarmos de qubits em superposição, lembre-se da moeda quântica girando no ar!

Em [03-mais-sobre-Hadamard.md](#) existe um experimento físico feito com fótons que ilustra muito bem esse conceito de superposição e medição quântica. Recomendo fortemente a leitura para aprofundar sua compreensão, caso esteja curioso! Somente me convenci intuitivamente disso após ler aquele experimento.

O que acontece: quando aplicamos uma porta Hadamard (veremos depois) e colocamos o qubit em um estado de superposição (como o estado $|+\rangle$), ele passa a ter 50% de chance de ser medido como 0 e 50% de chance de ser medido como 1. Ao fazer a medição, ele ‘colapsa’ (faz parar) essa superposição em um dos estados possíveis.”

Daí sempre precisamos realizar múltiplas medições para termos uma boa ideia das probabilidades reais envolvidas. É como jogar a moeda quântica 1024 vezes - chamamos de 1024 shots - para ver quantas vezes ela cai em cara ou coroa! Uma hora pode ser 30% de cara, outra hora 70%. Em outro momento pode ser 33% de cara. Mas se você jogar 1024 vezes, verá que a média se aproxima dos 50% esperados. Você consegue entender que o fim do cálculo se baseia, na verdade, em probabilidades estatísticas, certo? Não é como somar $1 + 1 = 2$, onde o resultado é sempre o mesmo. Aqui, o resultado pode variar a cada experimento, mas a média de muitos experimentos converge para o valor esperado.

Observação: É padrão realizar, por exemplo, 1024 shots para construir um histograma de probabilidades e ver qual é a resposta mais provável.

Claro que em um computador clássico fazer cálculo ($1 + 1 = 2$) de forma quântica demoraria muito, e seria coisa de louco calcular probabilidades estatísticas para tudo. Porém, você vai ver mais a frente, que existem certos cálculos que um computador clássico não consegue fazer de forma eficiente, pois se conseguisse, não haveria necessidade de um outro tipo de computador, certo? E é aí que entra o computador quântico. Ele não precisa ‘calcular’ essas probabilidades uma a uma; ele manipula as ondas de probabilidade (amplitudes) usando superposição, entrelaçamento e interferência para que, no final, a resposta correta tenha a maior chance de ser medida. Mas isso é assunto para os próximos notebooks!

O que acontece: É explicado que para simular um sistema quântico de n qubits, um computador clássico precisa armazenar 2^n números complexos (as amplitudes). Isso cresce exponencialmente. Com apenas 64 qubits, seriam necessários cerca de 18 quintilhões de números complexos, o que excede a capacidade de armazenamento atual para um computador clássico (este sofre para calcular essas amplitudes)!

Assim, quando você ouvir dizer que os cálculos realizados em um computador clássico são determinísticos (a gente sabe onde vai dar o resultado se fizermos “na mão”), enquanto os cálculos em um computador quântico são probabilísticos (é provável que dê 50% para...), agora você entenderá o porquê!

A superposição não é ser A e B. É ter a potencialidade de ser A ou B, com uma probabilidade que nós podemos controlar.

17.6 Portas Quânticas: Transformando Estados

Portas quânticas são **transformações** que aplicamos aos qubits. Matematicamente, são **matrizes** que multiplicamos pelos vetores de estado.

Da mesma forma que em computação clássica temos portas lógicas (AND, OR, NOT) que manipulam bits, em computação quântica temos portas quânticas que manipulam qubits.

Analogia: Se um estado quântico é como uma formiga em cima de uma bola (Esfera de Bloch), uma porta quântica é como uma rotação que move a formiga para uma nova posição na bola.

17.6.1 Principais Portas Quânticas:

1. **Porta X (NOT quântico)** - Inverte $|0\rangle \rightarrow |1\rangle$ (como o NOT da computação clássica)
2. **Porta H (Hadamard)** - Cria superposição (a porta mais importante!)
3. **Portas de Fase (Z, S, T)** - Modificam a fase sem mudar probabilidades

Vamos explorar cada uma delas!

Importante: Vamos trabalhar com kets ao representar os estados quânticos, como $|0\rangle$ e $|1\rangle$, para manter a notação padrão da computação quântica. Lembre-se que por trás desses kets, existem vetores coluna que representam os estados matematicamente, só isso!

17.6.2 Porta X (NOT Quântico)

É a versão quântica do NOT clássico. Funciona assim: - Se o qubit está em $|0\rangle \rightarrow$ vira $|1\rangle$ - Se o qubit está em $|1\rangle \rightarrow$ vira $|0\rangle$

Se fossemos usar vetores, seria: - Se o qubit está em $\begin{bmatrix} 1 \\ 0 \end{bmatrix} \rightarrow$ vira $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ - Se o qubit está em $\begin{bmatrix} 0 \\ 1 \end{bmatrix} \rightarrow$ vira $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$

Matriz da Porta X:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Como funciona:

A função é simbolizada como $X| \rangle$, onde $| \rangle$ é o estado do qubit. Assim, é como se:

- em X você substitui o X pela matriz da porta X que é $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
- e o $|0\rangle$ pelo vetor correspondente a $|0\rangle$, que é $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$.

Então, juntando X e $|0\rangle$, temos $X|0\rangle$:

$$X|0\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Daqui é só continuar com o cálculo sobre a multiplicação de matrizes para ver o resultado final:

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

E você percebe com que $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ se parece? Sim! É o vetor correspondente a $|1\rangle$. Então, concluímos que:

$$X|0\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} = |1\rangle$$

E para $X|1\rangle$?

$$X|1\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = |0\rangle$$

Dica: Multiplicação de matrizes é só “linha $\times$ coluna e somar”!

Vamos testar a porta X na prática com Python!

Definir a matriz da porta X

```
X = Matrix([[0, 1],  
            [1, 0]])
```

```
print(" Aplicando a Porta X...\n")
```

Aplicar X ao estado $|0\rangle$

```
result_0 = X * ket_0          # ket_0 é o estado  $|0\rangle$  com o vetor definido no código lá atrás, lembra?
```

```
print("Teste 1: X $|0\rangle$  deve dar  $|1\rangle$ ")
```

```
display(Markdown(f"$$$X \\ket{{{0}}} = \{{sp.latex(result_0)}\}$$$ "))
```

```
# Aplicar X ao estado |1
result_1 = X * ket_1
print("\nTeste 2: X|1 deve dar |0 ")
display(Markdown(f"$$$X \\ket{{{1}}} = {sp.latex(result_1)}$$$ "))

print("\n A porta X realmente inverte os estados, como esperado!")
```

Aplicando a Porta X...

Teste 1: $X|0\rangle$ deve dar $|1\rangle$

$$X|0\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Teste 2: $X|1\rangle$ deve dar $|0\rangle$

$$X|1\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

A porta X realmente inverte os estados, como esperado!

Aplicando a Porta X...

Teste 1: $X|0\rangle$ deve dar $|1\rangle$

Teste 2: $X|1\rangle$ deve dar $|0\rangle$

A porta X realmente inverte os estados, como esperado!

```
# Agora vamos definir a porta X usando NumPy (outra maneira de fazer a mesma coisa)

gate_x = np.array([
    [0, 1], # Primeira linha da matriz
    [1, 0] # Segunda linha da matriz
])

print("Porta X definida com NumPy:")
print(gate_x)
print("\n Pronta para usar em cálculos numéricos!")
```

Porta X definida com NumPy:

```
[[0 1]
 [1 0]]
```

Pronta para usar em cálculos numéricos!

Porta X definida com NumPy:

```
[[0 1]
 [1 0]]
```

Pronta para usar em cálculos numéricos!

```
# Vamos aplicar a porta X ao estado |0 e ver o resultado numérico

print("=== Aplicando Porta X ao estado |0 ===\n")

# Multiplicação de matriz por vetor: np.dot(matriz, vetor)
novo_estado = np.dot(gate_x, q_zero)

print("Estado inicial |0:")
print(q_zero)

print("\nApós aplicar a porta X:")
print(novo_estado)

print("\n Esperávamos |1 (vetor [0, 1]), e foi isso que obtivemos!")
print(" A porta X funcionou como um 'interruptor' que inverte o qubit!")
```

=== Aplicando Porta X ao estado |0 ===

Estado inicial |0 :

```
[[1]
 [0]]
```

Após aplicar a porta X:

```
[[0]
 [1]]
```

Esperávamos |1 (vetor [0, 1]), e foi isso que obtivemos!

A porta X funcionou como um 'interruptor' que inverte o qubit!

=== Aplicando Porta X ao estado |0 ===

Estado inicial |0 :

```
[[1]
 [0]]
```

Após aplicar a porta X:

```
[[0]
 [1]]
```

Esperávamos |1 (vetor [0, 1]), e foi isso que obtivemos!

A porta X funcionou como um 'interruptor' que inverte o qubit!

17.6.3 Porta H (Hadamard)

A porta H, ou porta Hadamard, é representada pela seguinte matriz:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Como funciona:

Vamos aplicar a porta H ao estado |0 :

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \cdot 1 + 1 \cdot 0 \\ 1 \cdot 1 + (-1) \cdot 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Agora, por que o resultado $\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ vai ser igual a $\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$?

Isso ocorre porque o vetor $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ pode ser expresso como a soma dos vetores base:

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = |0\rangle + |1\rangle$$

Portanto:

$$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

Esta expressão demonstra o efeito fundamental da porta Hadamard: ela transforma um estado definido ($|0\rangle$) em uma **superposição** com amplitudes iguais. Os coeficientes $\frac{1}{\sqrt{2}} \approx 0.707$ garantem a normalização, ou seja, $\left(\frac{1}{\sqrt{2}}\right)^2 + \left(\frac{1}{\sqrt{2}}\right)^2 = 1$.

Vamos agora aplicar a porta H ao estado $|1\rangle$:

$$H|1\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \cdot 0 + 1 \cdot 1 \\ 1 \cdot 0 + (-1) \cdot 1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle$$

Note que a porta Hadamard é **auto-adjunta** (igual à sua transposta conjugada), propriedade fundamental das portas quânticas reversíveis.

```
# Exemplo de aplicação da porta H (Hadamard)

# Definir a porta H
H = (1 / sp.sqrt(2)) * Matrix([[1, 1],
                               [1, -1]])

# Aplicar a porta H ao estado |0
result_h0 = H * ket_0

# Aplicar a porta H ao estado |1
result_h1 = H * ket_1

# Explicar o resultado utilizando Markdown para formatação bonita (perfumaria)

print("=== Aplicando Porta H (Hadamard) ===\n")
print("A porta H cria superposição dos estados |0 e |1.\n")
display(Markdown(f"$$$H \ket{{0}} = {sp.latex(H)} \cdot {sp.latex(ket_0)} = {sp.latex(result_h0)}$$$"))

print("\nResultado detalhado:")
display(Markdown(f"$$$ {sp.latex(result_h0)} = \frac{{1}}{{\sqrt{{2}}}} \ket{{0}} + \frac{{1}}{{\sqrt{{2}}}} \ket{{1}}$$$"))

print("\nAgora aplicando H ao estado |1:\n")
display(Markdown(f"$$$H \ket{{1}} = {sp.latex(H)} \cdot {sp.latex(ket_1)} = {sp.latex(result_h1)}$$$"))

print("\nResultado detalhado:")
display(Markdown(f"$$$ {sp.latex(result_h1)} = \frac{{1}}{{\sqrt{{2}}}} \ket{{0}} - \frac{{1}}{{\sqrt{{2}}}} \ket{{1}}$$$"))

=== Aplicando Porta H (Hadamard) ===
```

A porta H cria superposição dos estados $|0\rangle$ e $|1\rangle$.

$$H|0\rangle = \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix}$$

Resultado detalhado:

$$\begin{bmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix} = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

Agora aplicando H ao estado $|1\rangle$:

$$H|1\rangle = \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} \end{bmatrix}$$

Resultado detalhado:

$$\begin{bmatrix} \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} \end{bmatrix} = \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle$$

=== Aplicando Porta H (Hadamard) ===

A porta H cria superposição dos estados $|0\rangle$ e $|1\rangle$.

Resultado detalhado:

Agora aplicando H ao estado $|1\rangle$:

Resultado detalhado:

```
# Outra maneira de definir a porta H usando NumPy (faz a mesma coisa que o código anterior)
# Porta H (Hadamard): Cria superposição
# Matriz: (1/√2) * [[1, 1], [1, -1]]
gate_h = (1 / np.sqrt(2)) * np.array([
    [1, 1],
    [1, -1]
])

# APLICANDO A PORTA HADAMARD em |0>
# Operação: H * |0> = Superposição
estado_superposicao = np.dot(gate_h, q_zero)

print("\n--- Após aplicar a Porta Hadamard (H) ---")
print(f"Estado de Superposição:\n{estado_superposicao}")
# Note que os valores são aprox 0.707 (que é 1/raiz(2))
```

--- Após aplicar a Porta Hadamard (H) ---

Estado de Superposição:

```
[[0.70710678]
 [0.70710678]]
```


--- Após aplicar a Porta Hadamard (H) ---
 Estado de Superposição:
 $\begin{bmatrix} 0.70710678 \\ 0.70710678 \end{bmatrix}$

A Superposição

Quando aplicamos a `gate_h`, o resultado foi:

$$\begin{bmatrix} 0.707... \\ 0.707... \end{bmatrix}$$

Isso significa que o qubit não é nem 0 e nem 1. Ele contém “informação” de ambos. Matematicamente, ele é:

$$|\psi\rangle = 0.707|0\rangle + 0.707|1\rangle$$

Se você ainda não entendeu completamente o conceito de fase quântica, talvez fica melhor de visualizar na Esfera de Bloch, que veremos em [00b-phases.ipynb](#)! Depois volte para cá para reforçar o entendimento.

17.6.4 Produto interno (Inner Product) ou “bra-ket”

O produto interno entre dois estados quânticos $|\psi\rangle$ e $|\varphi\rangle$ é denotado por $\langle\psi|\varphi\rangle$ e é calculado como:

Ket é o vetor coluna que representa o estado quântico, enquanto bra é o vetor linha conjugado transposto correspondente.

Como se calcula o vetor linha conjugado transposto? Para calcular o vetor linha conjugado transposto (bra) de um vetor coluna (ket), você deve seguir dois passos: 1. Transpor o vetor coluna, ou seja, transformar suas linhas em colunas. 2. Tomar o conjugado complexo de cada elemento do vetor transposto, o que significa substituir cada número complexo pelo seu conjugado (inverter o sinal da parte imaginária).

Se $|\psi\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$ e $|\varphi\rangle = \begin{bmatrix} \gamma \\ \delta \end{bmatrix}$, então:

$$\langle\psi|\varphi\rangle = [\alpha \quad \beta] * \begin{bmatrix} \gamma \\ \delta \end{bmatrix} = \alpha * \gamma + \beta * \delta$$

Exemplo de produto interno

```
bra_0 = ket_0.T
bra_1 = ket_1.T
```

```
# 0|0
phi_00 = bra_0 * ket_0
```

```
# 0|1
phi_01 = bra_0 * ket_1
```

```
# 1|0
phi_10 = bra_1 * ket_0
```

```
# 1|1
phi_11 = bra_1 * ket_1
```

```
display(Markdown(f"$$$\\langle 0 | 0 \\rangle = {sp.latex(bra_0)} * {sp.latex(ket_0)} = {phi_00[0,0]}$$$"))
```

```
display(Markdown(f"$$\\langle 0 | 1 \\rangle = {sp.latex(bra_0)} * {sp.latex(ket_1)} = {phi_01[0,0]}$$"))
display(Markdown(f"$$\\langle 1 | 0 \\rangle = {sp.latex(bra_1)} * {sp.latex(ket_0)} = {phi_10[0,0]}$$"))
display(Markdown(f"$$\\langle 1 | 1 \\rangle = {sp.latex(bra_1)} * {sp.latex(ket_1)} = {phi_11[0,0]}$$"))
```

$$\langle 0|0\rangle = \begin{bmatrix} 1 & 0 \end{bmatrix} * \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1$$

$$\langle 0|1\rangle = \begin{bmatrix} 1 & 0 \end{bmatrix} * \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0$$

$$\langle 1|0\rangle = \begin{bmatrix} 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 0$$

$$\langle 1|1\rangle = \begin{bmatrix} 0 & 1 \end{bmatrix} * \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 1$$

17.6.5 Produto tensorial

O produto tensorial (ou produto de Kronecker) é uma operação matemática que combina dois estados quânticos para formar um estado composto. Se $| \psi \rangle$ e $| \phi \rangle$ são dois estados quânticos, o produto tensorial é denotado por $| \psi \rangle \otimes | \phi \rangle$.

Exemplo:

$$|0\rangle \otimes |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ 0 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

```
# Exemplo de produto tensorial

# |0> |0>
tensor_00 = sp.kronecker_product(ket_0, ket_0)
display(
    Markdown(
        f"$$\\ket{{0}} \\otimes \\ket{{0}} = {sp.latex(ket_0)} \\otimes {sp.latex(ket_0)} = \\begin{{bma"
    )
)

# |0> |1>
tensor_01 = sp.kronecker_product(ket_0, ket_1)
display(
    Markdown(
        f"$$\\ket{{0}} \\otimes \\ket{{1}} = {sp.latex(ket_0)} \\otimes {sp.latex(ket_1)} = \\begin{{bma"
    )
)

# |1> |0>
tensor_10 = sp.kronecker_product(ket_1, ket_0)
display(
    Markdown(
        f"$$\\ket{{1}} \\otimes \\ket{{0}} = {sp.latex(ket_1)} \\otimes {sp.latex(ket_0)} = \\begin{{bma"
    )
)
```

```

)

# |1    |1
tensor_11 = sp.kronecker_product(ket_1, ket_1)
display(
    Markdown(
        f"$$\\ket{{{1}}} \\otimes \\ket{{{1}}} = {sp.latex(ket_1)} \\otimes {sp.latex(ket_1)} = \\begin{{bma
    )
)

```

$$|0\rangle \otimes |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ 0 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = |00\rangle$$

$$|0\rangle \otimes |1\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\ 0 \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = |01\rangle$$

$$|1\rangle \otimes |0\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ 1 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = |10\rangle$$

$$|1\rangle \otimes |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\ 1 \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = |11\rangle$$

17.7 Representação de dois qubits

- Estado $|00\rangle$: $\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$
- Estado $|01\rangle$: $\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$
- Estado $|10\rangle$: $\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$
- Estado $|11\rangle$: $\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$

Dois qubits podem estar em superposição de todos os quatro estados possíveis:

$$|\psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle$$

onde $|\alpha|^2 + |\beta|^2 + |\gamma|^2 + |\delta|^2 = 1$.

17.7.1 Porta Controlled NOT (CNOT)

A porta CNOT (Controlled NOT) é uma porta lógica quântica que atua em dois qubits: um qubit de controle e um qubit alvo. A operação da porta CNOT é a seguinte:

- Se o qubit de controle estiver em $|0\rangle$, o qubit alvo permanece inalterado.
- Se o qubit de controle estiver em $|1\rangle$, o qubit alvo é invertido (0 vira 1 e 1 vira 0).

A porta CNOT é representada pela seguinte matriz:

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Quando aplicamos a porta CNOT a um estado composto de dois qubits, o resultado depende do estado do qubit de controle. Por exemplo, se aplicarmos a porta CNOT ao estado $|10\rangle$ (onde o primeiro qubit é o controle e o segundo é o alvo), o resultado será $|11\rangle$, pois o qubit de controle está em $|1\rangle$, então o qubit alvo é invertido.

Por exemplo:

$$\text{CNOT}|11\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} * \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = |10\rangle$$

```
# Implementação do uso da porta CNOT

# Definindo a porta CNOT
CNOT = np.array([
    [1, 0, 0, 0],
    [0, 1, 0, 0],
    [0, 0, 0, 1],
    [0, 0, 1, 0]
])

# |11>

q_11 = np.array([
    [0],
    [0],
    [0],
    [1]
])

# Aplicando a porta CNOT em |11>
novo_estado_cnot = np.dot(CNOT, q_11)
print("\n--- Após aplicar a Porta CNOT em |11> ---")
print(f"Esperamos |10>, obtivemos:\n{novo_estado_cnot}")

# Aplicando a porta CNOT em |10>
q_10 = np.array([
    [0],
    [0],
    [1],
    [0]
])
```

```

    [0]
])

novo_estado_cnot_10 = np.dot(CNOT, q_10)
print("\n--- Após aplicar a Porta CNOT em |10> ---")
print(f"Esperamos |11>, obtivemos:\n{novo_estado_cnot_10}")

```

```

--- Após aplicar a Porta CNOT em |11> ---
Esperamos |10>, obtivemos:

```

```

[[0]
 [0]
 [1]
 [0]]

```

```

--- Após aplicar a Porta CNOT em |10> ---
Esperamos |11>, obtivemos:

```

```

[[0]
 [0]
 [0]
 [1]]

```

```

--- Após aplicar a Porta CNOT em |11> ---
Esperamos |10>, obtivemos:

```

```

[[0]
 [0]
 [1]
 [0]]

```

```

--- Após aplicar a Porta CNOT em |10> ---
Esperamos |11>, obtivemos:

```

```

[[0]
 [0]
 [0]
 [1]]

```

17.7.2 Porta Controlled Z (CZ)

A porta CZ (Controlled Z) é uma porta lógica quântica que atua em dois qubits: um qubit de controle e um qubit alvo. A operação da porta CZ é a seguinte:

- Se o qubit de controle estiver em $|0\rangle$, o qubit alvo permanece inalterado.
- Se o qubit de controle estiver em $|1\rangle$, a fase do qubit alvo é invertida (multiplicada por -1).

A porta CZ é representada pela seguinte matriz:

$$CZ = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$$

Quando aplicamos a porta CZ a um estado composto de dois qubits, o resultado depende do estado do qubit de controle. Por exemplo, se aplicarmos a porta CZ ao estado $|11\rangle$ (onde o primeiro qubit é o controle e o